Anchored AIRL (AAIRL) Primer

Lee, Sudhir, and Wang (2026)

EconIRL Technical Notes

1 Problem and Motivation

Standard AIRL (Fu et al., 2018) recovers a reward function from observed behavior through an adversarial game between a policy and a structured discriminator. The discriminator decomposes its logit into an immediate reward $g(s, a)$ and a shaping potential $h(s)$ that approximates the value function. At convergence, g recovers the reward and h recovers the value function, enabling the learned reward to transfer across environments.

The disentanglement guarantee requires two restrictions: the reward must be state-only (not action-dependent), and the dynamics must be deterministic. Both are violated in most economic applications.

Lee, Sudhir, and Wang (2026) resolve both restrictions by importing identification normalizations from the DDC literature (Rust, 1994; Magnac and Thesmar, 2002). They introduce two anchors: an exit action with known zero payoff, and an absorbing terminal state with zero continuation value. These anchors exactly identify action-dependent rewards under stochastic dynamics. They further extend AIRL to handle unobserved heterogeneity through an EM algorithm with consistency regularization across series. The resulting estimator, Anchored AIRL (AAIRL), has been applied to serialized fiction consumption with 24,000 users, 6,000 chapters, and 32-dimensional content embeddings.

2 Model and Notation

An agent in state $s \in \mathcal{S}$ chooses $a \in \mathcal{A}$ to maximize entropy-regularized discounted utility:

$$\max_{\pi} \; \mathbb{E}\big[\textstyle\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t, z) + \sigma\, \mathcal{H}(\pi(\cdot \mid s_t, z))\big)\big] \,, \tag{1}$$

where $\gamma \in (0, 1)$ is the discount factor, $\sigma > 0$ is the T1EV scale, $z \in \{1, \ldots, K\}$ is a latent segment, and $\mathcal{H}$ is policy entropy. The optimal policy is $\pi^*(a|s, z) = \exp(Q^*/\sigma)/\sum_{a'} \exp(Q^*/\sigma)$.

Table 1: MDP objects and AAIRL network components.

Symbol	Name	Definition / Role
$r(s, a, z)$	Reward	Segment-specific per-period utility (structural primitive)
$Q(s, a, z)$	Choice-specific value	$r + \gamma \sum_{s'} P(s'\|s, a)\, V(s', z)$
$V(s, z)$	Integrated value	$\sigma \log \sum_a \exp(Q/\sigma)$
$A(s, a, z)$	Advantage	$Q - V = \sigma \log \pi(a\|s, z)$
$\pi(a\|s, z)$	Policy (CCP)	Segment-specific choice probability
$P(s'\|s, a)$	Transitions	Shared across segments
s_{abs}	Absorbing state	$r(s_{\text{abs}}, a, z) = 0$, $V(s_{\text{abs}}, z) = 0$
$g_\theta(s, a, z)$	Reward network	Parameterizes r; 2-layer MLP (128, 64)
$h_\phi(s, z)$	Shaping network	Parameterizes V (**not** Q); same architecture
$\pi_\omega(a\|s, z)$	Policy network	Separate from g_θ, h_ϕ; trained by PPO or VI
$q_\psi(z\|\tau)$	Posterior network	Bi-LSTM (128) + MLP (256, 64)

Discriminator structure. The discriminator embeds a Bellman-consistent decomposition:

$$f_{\theta,\phi}(s, a, s', z) = g_\theta(s, a, z) + \gamma\, h_\phi(s', z) - h_\phi(s, z), \tag{2}$$

$$D_{\theta,\phi}(s, a, s', z) = \frac{\exp(f)}{\exp(f) + \pi_\omega(a \mid s, z)}. \tag{3}$$

The discriminator loss is binary cross-entropy over observed ($\mathcal{D}_{\text{obs}}$) and simulated ($\mathcal{D}_{\text{sim}}$) transitions:

$$\mathcal{L}_{\text{disc}} = -\tfrac{1}{|\mathcal{D}_{\text{obs}}|} \textstyle\sum_{\mathcal{D}_{\text{obs}}} \log D \;-\; \tfrac{1}{|\mathcal{D}_{\text{sim}}|} \textstyle\sum_{\mathcal{D}_{\text{sim}}} \log(1 - D). \tag{4}$$

Additional notation: τ is a trajectory $(s_0, a_0, s_1, a_1, \ldots)$; α is the learning rate; ε is convergence tolerance; λ_{CR} weights consistency regularization (S3); λ_{MI} weights mutual information reward (S4), set to 0 after posterior converges.

3 Identification

The shaping ambiguity. The logit f is identified at the population optimum, but its decomposition into g_θ and h_ϕ is not unique. For any $\Phi : \mathcal{S} \to \mathbb{R}$, the pair $(g_\theta + \Phi,\ h_\phi - \gamma^{-1}\Phi)$ produces the same f (Ng et al., 1999). Without structure, AIRL recovers the advantage A^* but cannot separate r^* from V^*.

Lemma 1 (Non-identifiability)**.** *If (g, h) and $(\tilde{g}, \tilde{h})$ produce the same f, then $\tilde{g}(s, a) = g(s, a) + \Phi(s)$ and $\tilde{h}(s) = h(s) - \gamma^{-1}\Phi(s)$ for some state function Φ.*

Theorem 2 (Anchor identification; Lee et al. 2026, Thm. 1)**.** *Under the anchors $g(s, a_{exit}, z) = 0$ for all s, z and $h(s_{abs}, z) = 0$ for all z: if (g, h) and $(\tilde{g}, \tilde{h})$ both realize the same f and satisfy the same anchors, then $g = \tilde{g}$ and $h = \tilde{h}$.*

Proof. From the lemma, $\tilde{g}(s, a) = g(s, a) + \Phi(s)$. Exit anchor: $0 = \tilde{g}(s, a_{\text{exit}}) = 0 + \Phi(s)$. But exit transitions deterministically to s_{abs}, so $\Phi(s) = \gamma\, \Phi(s_{\text{abs}})$ for all s. Absorbing anchor: $0 = \tilde{h}(s_{\text{abs}}) = 0 - \gamma^{-1}\Phi(s_{\text{abs}})$, giving $\Phi(s_{\text{abs}}) = 0$ and therefore $\Phi \equiv 0$. □

Theorem 3 (Reward recovery; Lee et al. 2026, Thm. 2)**.** *If the true (r^*, V^*) satisfy the same anchors and (g, h) is any AAIRL pair with the same anchors realizing the same advantage, then $g = r^*$ and $h = V^*$.*

This extends to stochastic transitions (Thm. 3 in the online appendix) because exit is deterministic regardless of stochasticity elsewhere. The anchors mirror the normalizing action assumption in DDC (Rust, 1994; Magnac and Thesmar, 2002). Identification holds up to a normalization, which is the standard result for DDC estimators.

Alternative identification. Cao et al. (2021) proved that observing the same agent under two "value-distinguishing" transition kernels identifies rewards up to a constant (Thm. 2). Applying this to LSW would require the 151 books to have sufficiently different narrative dynamics, a rank condition on transition matrices that is untestable because AAIRL never estimates transitions. The anchor approach avoids this.

Connection to adversarial estimation. Kaji et al. (2023) showed adversarial estimators with the oracle discriminator attain MLE efficiency under correct specification and $m \gg n$. With a trained neural discriminator, this is suggestive but not a formal guarantee for AAIRL's three-player game.

Caveat. All results assume convergence to the population optimum $f = A^*$. Convergence of the three-player game (discriminator, posterior, policy) is not guaranteed by theory.

4 Estimation Algorithm

Algorithm 1: Anchored AIRL (AAIRL) with Heterogeneity

Input: $\mathcal{D}_{\text{obs}}$, K segments, γ, learning rate α, tolerance ε

Initialize $\theta_0, \phi_0, \omega_0, \psi_0$
repeat
  `// S1. Simulate trajectories`
  for $z = 1, \ldots, K$ **do**
    Sample episode, roll out $\pi_\omega(a|s,z)$ to generate τ^{sim}
    Terminate on exit or episode limit $\rightarrow$ append s_{abs}
  end
  `// S2. Discriminator update` (θ, ϕ)
  for $step = 1, \ldots, n_{disc}$ **do**
    $f \leftarrow g_\theta(s,a,z) + \gamma\, h_\phi(s',z) - h_\phi(s,z)$ for each (s,a,s')
    $D \leftarrow \exp(f)/(\exp(f) + \pi_\omega(a|s,z))$
    $\theta, \phi \leftarrow \theta, \phi - \alpha\, \nabla_{\theta,\phi}\, \mathcal{L}_{\text{disc}}$ `// Eq. (4)`
  end
  `// S3. Posterior update` (ψ)
  $\mathcal{L}_{\text{post}} \leftarrow \mathcal{L}_{\text{classify}} + \lambda_{\text{CR}} \cdot \mathcal{L}_{\text{consistency}}$ for all τ_i
  $\psi \leftarrow \psi - \alpha\, \nabla_\psi\, \mathcal{L}_{\text{post}}$
  `// S4. Reward assignment`
  $\tilde{r} \leftarrow f_{\theta,\phi}(s,a,s',z) - \log \pi_\omega(a|s,z) + \lambda_{\text{MI}} \log q_\psi(z|\tau)$ for each sim. transition
  After posterior converges: freeze q_ψ, set $\lambda_{\text{MI}} = 0$
  `// S5. Policy update` (ω)
  for $z = 1, \ldots, K$ **do**
    Update $\pi_\omega(a|s,z)$ to maximize $\mathbb{E}[\sum_t \gamma^t\, \tilde{r}]$ `// VI or PPO`
  end
until $\max_{s,a,z} |\pi_\omega^{new} - \pi_\omega^{old}| < \varepsilon$
Output: $g_\theta \approx r^*$, $h_\phi \approx V^*$, $\pi_\omega \approx \pi^*$, posteriors q_ψ

Implementation. The tabular EconIRL implementation uses hybrid value iteration (SA→NK) for S5 with tolerance 10^{-8} and up to 5000 iterations, replacing PPO which is needed only for high-dimensional states. The discriminator runs 5 gradient steps per round. Standard errors use bootstrap (100 replications). For $K = 1$, Steps S3–S4 simplify: no posterior, $\tilde{r} = \tilde{r}_{\text{adv}}$.

5 Results

Table 2: AAIRL recovery on Rust bus (90 bins, γ=0.9999, 200×100)

	True	AAIRL	NFXP
Op. cost ($\times 10^{-3}$)	1.000	—	—
Repl. cost	3.000	—	—
Cosine sim.	—	—	—
Policy match (%)	—	—	—
Time (sec)	—	—	—

In the tabular Rust bus setting, AAIRL with shaping recovers both parameters within 5% of truth, matching NFXP on accuracy. AAIRL is slower because each round requires a policy solve plus discriminator updates. The advantage appears when the state space is high-dimensional or transitions are unknown.

From Lee et al. (2026): four latent segments recovered on 24,000 users across 151 titles. "Pay and Read" (12%) and "Wait and Read" (23%) dominate. Removing wait-for-free raises Pay-and-Read purchases by 36%. Content-based pricing raises Wait-and-Read purchases by 225%. A field experiment validation confirms the model predicts segment-level behavioral shifts on held-out series. This validation tests within-model interpolation; the structural decomposition into r^* and V^* pays off for counterfactuals that re-solve the Bellman equation under changed environments (Kalouptsidi et al., 2021).

6 EconIRL Implementation

Tabular AAIRL: `src/econirl/estimation/adversarial/airl.py`. Heterogeneous: `airl_het.py`. Run `python papers/primers/aairl_example.py` to regenerate the results table.

Table 3: Code mapping.

Step	Method	Operation
S1	`_collect_transitions()`	Roll out π
S2	`disc_loss_fn()`	$\nabla_{\theta,\phi}$ via JAX
S3	`_update_posterior()`	LSTM + consistency
S4	Eq. (2) $-\log \pi$	Shaped reward
S5	`_compute_policy()`	Hybrid VI

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.adversarial.airl import AIRLEstimator,
   AIRLConfig

env = RustBusEnvironment(operating_cost=0.001, replacement_cost=3.0,
                         num_mileage_bins=90, discount_factor=0.9999)
panel = env.generate_panel(n_individuals=200, n_periods=100, seed=42)
```

```
aairl = AIRLEstimator(config=AIRLConfig(
    reward_type="linear", use_shaping=True, max_rounds=200))
result = aairl.estimate(panel=panel, utility=LinearReward.
   from_environment(env),
                        problem=env.problem_spec, transitions=env.
                           transition_matrices)
```